

An open-source cloud-based procedure for MODIS remote sensing products: The nasawebservicepython package

Alessandro Montaghi^{*}, Simone Bregaglio, Sofia Bajocco

Council for Agricultural Research and Economics (CREA) – Research Centre for Agriculture and Environment, Rome, Italy

ARTICLE INFO

Keywords:

Smart farming
Open-source software
Python
MODIS
Cloud-application

ABSTRACT

Living in the era of “big data” demands an increase in the application and development of solutions for data ingestion in agroecological research, both in the private and public sectors. Among available data sources, satellite imagery is a viable solution to acquire large amounts of data at cheaper costs. Although cloud-based commercially distributed services are flourishing, open-source software able to assist practitioners and researchers in downloading and pre-processing satellite imagery is strongly demanded by the remote sensing community. This is especially true when looking at applications developed in Python, a programming language that quickly gained popularity in agroecological science. In this letter, we introduce *nasawebservice* version 1.0.0, a Python Package designed to automatically collect remotely sensed data from the NASA MODIS/VIIRS Land Products web service. The *nasawebservice* package contains a set of classes and methods to facilitate download operations for practitioners and researchers and to implement user-friendly data ingestion pipelines in local and cloud environments. The main advantage of *nasawebservice* is the reduction of pre-processing operations to use satellite images due to getting the data in JSON format, with consequent saving of computational time for large data download and analytic operational pipelines.

1. Introduction

1.1. The rising availability of remote sensing data

Smart Farming, or Smart Agriculture, has become a new paradigm in modern agriculture production (Verdouw et al., 2021). Distinctive traits of Smart Farming are, on the one side, increasing qualitative and quantitative aspects of agricultural production and on the other side, reducing farmers' costs for management operations (Moysiadis et al., 2021). Smart Farming can be considered an extension of the Precision Agriculture concept (Kamilaris et al., 2016), as it explicitly integrates several Information and Communication Technology (ICT) solutions into farmers' production and supply chains (Virk et al., 2020).

Smart Farming Systems need to be implemented by a wide range of data from static and real-time information to perform to their full potential. Different and alternative data sources can be used for the purpose of delivering support to farmers, technicians and decision-makers in general by means of predictions, simulations, and plans. Some examples include Internet of Things (IoT) platforms (Jayaraman et al., 2016), Global Positioning Systems (GPS) devices connected to

agricultural machinery (Bacco et al., 2018), imagery sensors on board of Unmanned Aerial Vehicles (UAVs) (Lottes et al., 2017) or Unmanned Terrestrial Vehicles (UTVs) (Vu et al., 2018), in-situ weather stations or from public network services (Tenzin et al., 2017), including weather forecasts (Padalalu et al., 2017), digital devices and smart-apps for field data collection (Xue-Fen et al., 2018), soil datasets for irrigation scheduling, geographical information systems (GIS) (Nugroho et al., 2017), and simulations from machine learning algorithms and process-based models (Zhai et al., 2020).

Despite this complex scenario, the effective adoption of Smart Farming is hampered by the economic sustainability of these solutions for most of the farms, agricultural cooperatives, and public administrations, which have to face high recurrent costs related to installation and maintenance of IoT devices, or to expensive subscriptions to services from private companies (Bellini et al., 2021). Among available sources of information, satellite imagery is the one providing more data at cheaper costs, because several Space Agencies, as a result of their missions, provide open satellite data as public services. Most of these services dispense their data products according to the guidelines of the REpresentational State Transfer (REST) approach, which has gained

^{*} Corresponding author.

E-mail address: alessandro.montaghi@crea.gov.it (A. Montaghi).

popularity in the networking protocol design, and it has become a prevalent choice for implementing web services (Tsalgatidou and Pilioura, 2002). Since 2000, when Roy Fielding presented his doctoral thesis about architectural styles and designs of network-based software architectures (Fielding and Richard, 2002), the REST-Calls have been employed throughout the ICT industry for creating reliable stateless web APIs (Application Programming Interfaces). For instance, users can access subsets of MODIS (MODerate-resolution Imaging Spectroradiometer, Justice et al., 1998) and VIIRS (Visible Infrared Imaging Radiometer Suite, Murphy et al., 2006) products through the MODIS/VIIRS Land Products Web Service of the National Aeronautics and Space Administration (NASA), without requiring an account credential. The NASA Web Service is based on REST web service API (OORN DAAC, 2018) and on the OpenAPI Specification (OAS 1.0), which provides API definitions in a language well-known to humans and computers. Regarding the European Space Agency (ESA), the data products of the European Union's Earth observation programme Copernicus are accessible through the Copernicus Open Access Hub (previously known as Sentinels Scientific Data Hub), which provides a dedicated API Hub service interface developed on Open Data Protocol (OData), a like-REST data access protocol, in order to promote free and open data access policy (Tona and Bua, 2018). A comprehensive description of the query options can be found on the Copernicus Open Access Hub web page. The United States Geological Survey (USGS), promoter together with NASA of the Landsat medium resolution Earth observation satellite mission (Wulder et al., 2019), provides a Machine-to-Machine (M2M) API service, or a JSON-based REST API protocol, in order to interact with USGS/EROS data inventories, to identify and retrieve HTTP download URLs for data or to submit bulk download orders.

One of the major challenges with remote sensing images is dealing with the data ingestion process, addressing the import, and pre-processing of multi-dimensional and heterogeneous satellite data (Liu, 2015). The increasing availability of satellite missions and high temporal and spatial resolution satellite imagery, and the growing dimension of the respective data archives, make traditional data processing approaches (e.g., using desktop applications) unfeasible and necessarily lead to a change in the design of data processing systems (Tamiminia et al., 2020). This implies the use of modern information technologies to automatize data ingestion, storage, analysis, and sharing, which exploit, for instance, cloud computing-based services. Indeed, by using the ever-increasing availability of cloud-based infrastructures and solutions offered by major IT industry players, such as Azure hosted by Microsoft, Amazon Web Services (AWS), and Google Cloud, new tailored cloud-based computing platforms may be created to harness the potential offered by the Big Data age (Rodríguez-Mazahua et al., 2016). In this regard, in the wake of what already developed in the "Smart Cities" domain (Bellini et al., 2020), the actual trend in the development of Smart Farming systems entails the development of efficient techniques for the ingestion of satellite data sources and their integration with other complementary data sources.

Research in agriculture and ecology requires efficient software pipelines to download and pre-process satellite imagery. Remote sensing datasets are used to analyze the biological diversity of agroecosystems through their structural, compositional, and functional measurements and have become essential geospatial input in various fields involving, e.g., species distribution modeling (Schwager and Berg, 2021), process-based yield forecasting (Bregaglio et al., 2023) and quantitative landscape ecology (Crowley and Cardille, 2020). The development of operational pipelines is often carried out by researchers with self-acquired ICT skills who leverage the tools implemented in Python or R language. Nonetheless, the download and pre-processing steps carried out by different research groups share the needs of having easy-to-use packages to speed up the data homogenization process, which in many cases involve the very same steps across disciplines.

1.2. Cloud-computing state-of-the-art

Currently, the most used cloud-based platform for geo-data processing is Google Earth Engine (GEE) launched by Google in 2010 (Gorelick et al., 2017). The success of GEE is due to its implementation and usage characteristics. GEE has the features of storing petabytes of remote sensing imagery (e.g., full archive of Landsat, Sentinel 1, 2, 3 and 5-P, and MODIS and its derived products), climate-weather data (e.g., global temperature and precipitation measurements), and many other geophysical and socio-economic publicly available datasets (e.g., digital elevation models, land cover maps, and spatial demographic layers). Furthermore, all data processing workflows are implemented in a large automatic parallel processing system that allows to effectively perform, in computational time, the data analysis (Gorelick et al., 2017). To these implementation features, GEE has provided an easy-to-use Earth Engine API for the users. A user can access the APIs service using a client library or through an appropriate web-based Integrated Development Environment (IDE; Gorelick et al., 2017). A more complete and detailed review of GEE, which is beyond the scope of this article, can be found in Amani et al. (2020) and Tamiminia et al. (2020).

Regardless of the undoubted scientific value of this cloud computing platform for satellite imagery and geospatial analysis, there are some drawbacks that hamper the migration of public and private services for GEE. In detail:

1. The GEE is a closed platform controlled by Google, a private US company. For any reason, regardless of the will of the user, the service could be interrupted, severely limited, or have future high usage costs. Although GEE is free for research, education, and nonprofit use a paid license is required to develop operational applications. Finally, the data are stored, and the workflows are processed on a private company's servers, which can be subject to issues related to data protection law, such as the European General Data Protection Regulation-GDPR (Maniadaki et al., 2021).
2. According to the cloud architecture, GEE is a SaaS (Software-as-a-service). This translates into services (e.g., available datasets, allocated resources, implemented algorithms, export data formats, and so on) entirely managed by Google and not (or hardly) modifiable by the single user. For instance, in GEE deep learning analysis can be performed using Tensorflow, a machine learning and artificial intelligence open-source framework developed by the Google Brain team (Abadi et al., 2016). Although Tensorflow is one of the leading machine learning frameworks, this excludes the possibility of employing different open-source machine learning systems such as PyTorch primarily developed by Facebook AI Research lab (FAIR) (Ketkar and Jojo, 2021), Microsoft Cognitive Toolkit (CNTK) (Seide and Amit, 2016), Caffe of the University of California, Berkeley, US (Jia et al., 2014) or novel algorithms for deep learning (Zhu et al., 2017). These evidences limit the possibility to build a highly customizable and scalable cloud infrastructure.
3. GEE is primarily a platform for geospatial analysis, which tends to encapsulate within itself the complexity of the processes. However, in the new context of Big Data analysis, there is the need to collect, aggregate and manage highly heterogeneous multidimensional data and metadata coming from several operators and the integration of thousands of different data models, as proposed by different vendors and/or providers, in order to establish relationships among data. Although the GEE public data archive includes several updated geospatial products, very often regional and local scale sources of information, such as agronomic data from extension services, weather stations or pedological databases, are absent in the GEE catalog. This information is fundamental in the domain of Smart Farming (Wolfert et al., 2017). As discussed by Bellini et al. (2020), additional aspects of Big Data analysis are related to semantic data aggregation, data redundancy management, fault tolerance handling, licenses of use and data protection, real-time queries on the

Knowledge Base, data analytic and visualization or GIS-based visualization, etc. All these issues are difficult or impossible to handle by developing IT infrastructures based on the GEE platform.

However, in addition to the development of GEE and other proprietary geospatial platforms such as Amazon “Earth on AWS” and Microsoft Planetary Computer, the development of free and open-source functionalities and components is crucial to allow transparency, flexibility, reproducibility, and free data access (Rocchini and Neteler, 2012). Based on the open-source concept, it is possible to create solutions for the management and processing of satellite data to be integrated in Smart Farming systems. This also ensures that the “openness” requirement of an open-source platform can be satisfied, i.e., the capability of adding new functionalities and modules, of interacting with any protocol and service (e.g., Rest Call, Web services, HTTP/HTTPS, and so on), of minimizing re-engineering efforts (Jayaraman et al., 2016), and of easily sharing and integrating own components with others (Bellini et al., 2022). Finally, by developing open-source systems of this kind, any interested user can download and install them on local and cloud platforms at no cost.

1.3. Python solutions

In this framework, several Python solutions have been proposed to acquire and ingest remote sensing data. Among high-level interpreted programming languages (e.g., R, JavaScript, Ruby, PHP), the general-purpose Python language has seen terrific success in scientific computing, data science, machine learning, and remote sensing data processing (Raschka et al., 2020). For the satellite data ingestion process, several Python packages have been devoted for interfacing with the USGS’s Inventory Service API (USGS library), providing a command-line interface or a Python API to search and download Landsat Collections scenes (landsatxplore package, Forget et al., 2018); searching, downloading and retrieving metadata of Sentinel satellite images from the ESA Copernicus Open Access Hub (Sentinelsat package, Wille et al., 2017); downloading, mosaicking, and reprojecting MODIS data (pyModis library, Delucchi, 2014), just to name a few. Nonetheless, all these packages currently provide data acquisition involving the local download of the file of the entire tile. As highlighted by Bellini et al. (2021), since each tile is the result of a satellite scan that covers a region that goes beyond the scope of the interest area (e.g., farm or parcel of cultivated land), it is very often necessary to pre-process and clean up these data from unnecessary locations and invalid values. Furthermore, due to the initial download, the data ingestion is a somehow slow process, requires a lot of memory for data storage, and is expensive in term of computational time for intermediate processing steps.

To overcome all drawbacks, in this article, we introduce the *nasawebservice* Python package, whose source code (version 1.0.0) is available, under GNU GPL (General Public License) v3.0, on GitHub at the following URL: <https://github.com/ammoniaca/nasawebservice>. A PyPI (Python Package Index) package for *nasawebservice* is also under development. The aim of this article is to present a cloud-based procedure for automatic MODIS data gathering using *nasawebservice*. The main advantage of our realization relies in the ease and fast data pre-processing steps of satellite data, being *nasawebservice* based on the APIs REST protocol of the NASA MODIS/VIIRS Land Products Web Service.

2. Methods

2.1. The NASA MODIS/VIIRS land products web service

The MODIS sensors aboard TERRA and AQUA provide twice-daily data since 2000 and 2002, respectively, with 250 m, 500 m, and 1 km spatial resolutions. The MODIS row data are composited to daily, 8-day, 16-day, and yearly imagery, and stored in the Land Processes

Distributed Active Archive Center (LP DAAC) (<https://lpdaac.usgs.gov>). These data are a unique resource of information for researchers working in the field of agriculture (e.g., Alcantara et al., 2012; Angearu et al., 2020; Xiao et al., 2006), ecology (e.g., Jepsen et al., 2009; Justice et al., 2002), and global change (e.g., Mao et al., 2017; Zhang et al., 2004). For this reason, NASA provides several web-based tools to retrieve and download MODIS data.

The NASA MODIS/VIIRS Land Products web service model, built on ORNL DAAC’s MODIS/VIIRS Global sub-setting and visualization tool, provides a user interface to retrieve MODIS and VIIRS Land Product customized subsets (<http://daac.ornl.gov/MODIS/MODISmenu/modiswebservice.html>). The web service uses standards based on REST APIs. The URIs (Uniform Resource Identifiers) syntax to address the service resources is <https://modis.ornl.gov/rst/api/{version}/{resourcepath}>, where the current version is v1 (2022-04-08). We can distinguish two types of resource paths: a Global Subset (.../api/v1/products) and a Fixed Sites (.../api/v1/sites) path. The Global Subset path exposes multiple API HTTP GET methods (i.e., products, bands, dates, subset, and subsetOrder) by which it provides several global MODIS/VIIRS land products and, only for North America, the Daymet daily surface weather data. For each API method, the current response content type can be chosen between text/csv (i.e., comma-separated values) or application/json format, following the common MIME (Multipurpose Internet Mail Extensions) types guidelines (Ned and Borenstein, 1996). A more complete and detailed review of Global Subset methods can be found on the Open API documentation page of the web service. On the contrary, the Fixed Sites APIs interface, not implemented in the current version of *nasawebservice*, provides data for selected fields and flux towers, belonging to a certain research network (e.g., FLUXNET Canada), across the world.

2.2. The Python *nasawebservice* package

The Python *nasawebservice* is a user-friendly package that provides a set of classes and methods to define pipelines for the automatic MODIS data ingestion from the NASA MODIS/VIIRS Land Products Web Service. The *nasawebservice* is implemented in the Python 3 programming language. Its dependencies are requests, aiohttp, and asyncio (Hunt, 2019). The remaining dependencies used are standard libraries for maximal compatibility. To facilitate the users in the creation of their data process workflows, the *nasawebservice* was designed to be as close as possible to the Global Subset APIs of the NASA service making use of wrapper methods. Furthermore, the library provides additional methods with the purpose of making the data ingestion pipeline scalable according to various user needs. All these methods are exposed by *NasaWebServiceAPIProvider* class. *NasaWebServiceAPIProvider* object is initialized by passing as arguments the URL base of NASA service (i.e., “<https://modis.ornl.gov>”) and the current version (default: *version* = 1).

Finally, to simplify the parameters setup of each present method, a user-defined data type paradigm has been implemented by using enumeration classes. An enumeration is a special class that represents a group of symbolic names bound to unique, constant values (i.e., unchangeable variables). For example, the *Products* enum class represents a collection of available product names in the service. The *Bands* class is a composition of enumeration bands classes available for each product. Table 1 shows the methods exposed by the *NasaWebServiceAPIProvider* with the input parameters type.

The wrapper methods implemented in the library are *products()*, *bands()*, *dates()*, and *subsets()*. The *products* method (default: *sensor* = *Sensors.ALL* and *tool* = *Tools.ALL*) returns a list of available products for all sensors. The *bands* and *dates* methods return the list of all available bands and the list of all available dates for each product, respectively. These methods allow the users to explore the NASA data archive and plan an efficient data ingestion. More specifically, Listings 1 and 2 show two examples of *nasawebservice* code to retrieve the list of products available, the list of bands available for a given product (e.g., Terra

Table 1

The methods exposed by *NasaWebServiceAPIProvider* class of *nasawebservice* package. Par(s) = parameters.

Method	Description and Parameters
products	Get all available products par(s): <i>sensor, tool</i>
bands	Get all available bands for a specific product par(s): <i>product</i>
dates	Returns the list of available dates par(s): <i>product, latitude, longitude</i>
subsets	Returns a data subset with a limit of 10 MODIS dates per request par(s): <i>product, latitude, longitude, band, start_date, end_date, km_above_below, km_left_right</i>
execute_many	Returns the subset without the limit 10 MODIS dates per request par(s): <i>product, latitude, longitude, band, start_date, end_date, km_above_below, km_left_right</i>
execute_all	Returns data for all available dates par(s): <i>product, latitude, longitude, band, km_above_below, km_left_right</i>
async_execute_many	Asynchronous version of execute_many par(s): <i>product, latitude, longitude, band, start_date, end_date, km_above_below, km_left_right</i>
async_execute_all	Asynchronous version of execute_all par(s): <i>product, latitude, longitude, band, km_above_below, km_left_right</i>
last_product	Returns the last imagery upload in the NASA server par(s): <i>product, latitude, longitude, band, km_above_below, km_left_right</i>
is_present	Checks if a given product is available for a reference date par(s): <i>product, latitude, longitude, reference_date</i>
available_products_by_date	Returns all available and not available products for a reference date par(s): <i>latitude, longitude, reference_date</i>

Glossary: *sensor* (Sensors object): collection of all available sensors (i.e., MODIS-Terra, MODIS-Aqua, MODIS-TerraAqua, VIIRS- SNPP, Daymet, SMAP, ECOSTRESS, and SIFESDR); *tool* (Tools object): collection of available tools (i.e., FixedSite and GlobalSubset); *product* (Products object): collection of all types of products; *band* (Bands object): collection of all available bands for each product; *latitude* (float) and *longitude* (float): decimal degrees in geographic latitude and longitude WGS 84 coordinate system; *start_date* (string) and *end_date* (string): start or end date of the subset period in the format AYYYYDDD; *reference_date* (string): reference date in the format AYYYYDDD; *km_above_below* (integer) and *km_left_right* (integer): size in km of the “subset above and below” or “left and right” the center location (latitude, longitude). The value must be between 0 and 100.

MODIS Vegetation Indices, MOD13Q1), the list of composite dates available for the product, latitude, longitude combination, and the subset for the location, product, band, and date combination.

Listing 1. Snippet of *nasawebservice* wrapper methods.

location on Earth in decimal degrees in WGS 84 coordinate system (EPSG: 4326), iii) a single band or all bands of the product, iv) the start and the end dates of the subset period, and v) the dimension of the area of interest around the location, defined as the distance in kilometers around the center location. Following the MODIS/VIIRS Web Service

```

1 from nasawebservice import *
2 from pprint import pprint
3 url = "https://modis.ornl.gov"
4 # Create a MODIS data provider
5 provider = NasaWebServiceAPIProvider(base_url=url, version=1)
6 # Get all NASA web service available products
7 all_products = provider.products(sensor=Sensors.ALL, tool=Tools.ALL)
8 # Get all Bands available for MOD13Q1 product
9 all_bands = provider.bands(product=Products.MOD13Q1)
10 # Get all dates available for MOD13Q1 in a given location (latitude, longitude)
11 all_dates = provider.dates(
12     product=Products.MOD13Q1,
13     latitude=44.04780019,
14     longitude=10.35481917
15 )
16 pprint(all_products)
17 pprint(all_bands)
18 pprint(all_dates)

```

The primary goal of *nasawebservice* is to facilitate the users in the MODIS data extraction, as JSON format, from the NASA web service. A service request can be created using the *subset()* method, requiring just i) the product type, ii) the latitude and longitude values for any center

specifications, the dates are expressed as string type formatted like “AYYYYDDD”, where A is a symbolic constant, YYYY is the year and DDD is the day of the year. In detail, the JSON file that returns from the execution of the *subset()* method, contains, for each date, an array data structure with the value of the MODIS/VIIRS cell corresponding to the center location or the cells included in the area of interest, plus various ancillary information (e.g., file name, tile, cell size, number of columns

and rows, and so on).

Listing 2. Snippet of *nasawebservice* subset wrapper method.

```

1 from nasawebservice import *
2 from pprint import pprint
3 provider = NasaWebServiceAPIProvider(base_url="https://modis.ornl.gov")
4 # Get all bands pixel value subset for MOD13Q1 from 2020-12-02 to 2021-04-03
5 mod13q1_subsets = provider.subset(
6     product= Products.MOD13Q1,
7     latitude= 44.04780019,
8     longitude=10.35481917,
9     band= Bands.MOD13Q1.ALL,
10    start_date="A2020337",
11    end_date= "A2021113",
12    km_above_below=0,
13    km_left_right=0
14 )
15 pprint(mod13q1_subsets)

```

execute_all() method, by which a user can get all available data in the NASA server for a given product and location. The call of the methods *execute_many*, *execute_all*, and their respective asynchronous versions,

async_execute_many and *async_execute_all* is reported in Listings 3 and 4.

Listing 3. Snippet of *nasawebservice* *execute_many* and *execute_all* methods.

According to the NASA specification, there is a limit of a maximum of ten dates per request. In order to avoid this limitation, the *execute_many* () method has been implemented. The *execute_many*() accepts the same

```

1 from nasawebservice import *
2 from pprint import pprint
3 provider = NasaWebServiceAPIProvider(base_url="https://modis.ornl.gov")
4 # Get NDVI pixel values for MODIS MOD13Q1 product from 2020-12-02 to 2021-04-23
5 mod13q1_ndvi_many = provider.execute_many(
6     product= Products.MOD13Q1,
7     latitude= 44.04780019,
8     longitude=10.35481917,
9     band= Bands.MOD13Q1.NDVI,
10    start_date="A2020337",
11    end_date= "A2022049",
12    km_above_below=0,
13    km_left_right=0
14 )
15 for item in mod13q1_ndvi_many: pprint(item)
16 # Get all NDVI pixel values
17 mod13q1_ndvi_all = provider.execute_all(
18     product= Products.MOD13Q1,
19     latitude= 44.04780019,
20     longitude=10.35481917,
21     band= Bands.MOD13Q1.NDVI,
22     km_above_below=0,
23     km_left_right=0
24 )
25 for item in mod13q1_ndvi_all: pprint(item)

```

input parameters of the *subset()* method, but returns a lazy iterator. When *nasawebservice* has to deal with large volume of requests, the choice to use an iterator improves the memory management for the data ingestion operations. The same approach has been pursued in the

Furthermore, two asynchronous method versions have been developed for *execute_many()* and *execute_all()*, namely *async_execute_many()* and *async_execute_all()*. These methods use the *aiohttp* library introduced in Python 3.4 to work with *asyncio*, and avoid the “Callback Hell” to

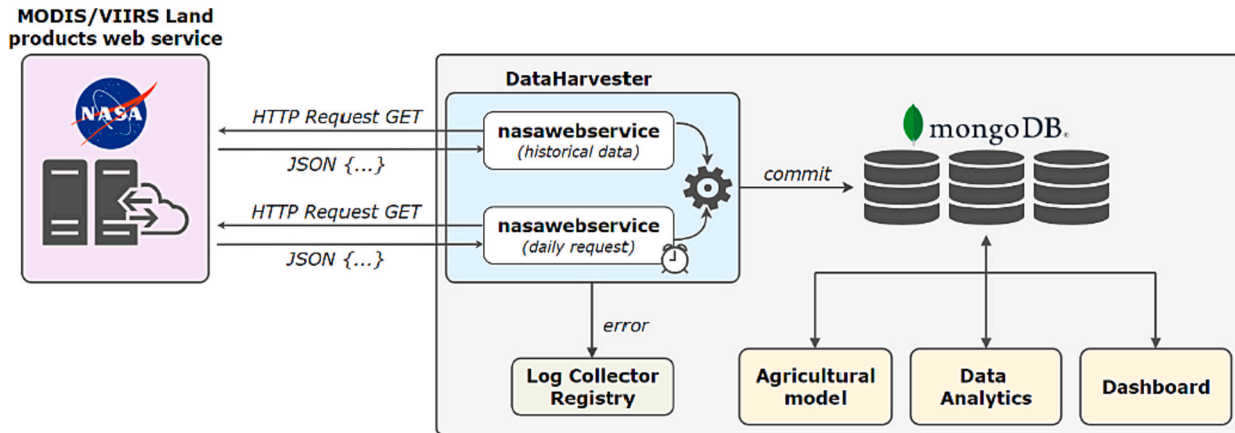


Fig. 1. Technical cloud-based architecture of the MODIS data ingestion and data flows: from NASA data source to *nasaweb service* processes, storage and agricultural models, data analytics and dashboard.

handle multiple concurrent requests in parallel at high speed (Kawaguchi and Ozawa, 2019). The latter methods have better performance in term of speed, at the cost of partial ordering of the response messages. Hence, when the responses arrival order is not critical, the asynchronous methods could be used to increase the performance. Otherwise, the overall time complexity of the ingestion phase shall also consider the time and space computational cost of the sorting algorithms. The implementation of additional asynchronous methods is planned for future releases.

Listing 4. Snippet of asynchronous version of *execute_many* and *execute_all* methods.

NASA's MODIS mission provides twice-daily global coverage. For this purpose, the *NasaWebServiceAPIProvider* class is equipped with specific methods to create daily pipelines for data ingestion (Listing 5). For instance, the *last_product()* method accepts as argument the product, location, and band combination, and returns the last imagery uploaded in the NASA server. The method *is_present()* allows the user specifying a reference date (e.g., the date of today) and receiving a Boolean response stating if a given product is present in the server archive. If the answer is "True" (i.e., the given product is in the archive), a *subset()* method request can be triggered to retrieve the data. Otherwise, the application waits a user-defined time before starting a new query. Moreover, the

```

1 from nasaweb service import *
2 from pprint import pprint
3 provider = NasaWebServiceAPIProvider(base_url="https://modis.ornl.gov")
4 # asynchronous version of execute_many method
5 async_mod13q1_ndvi_many = provider.async_execute_many(
6     product= Products.MOD13Q1,
7     latitude= 44.04780019,
8     longitude=10.35481917,
9     band= Bands.MOD13Q1.NDVI,
10    start_date="A2020337",
11    end_date= "A2022049",
12    km_above_below=0,
13    km_left_right=0
14 )
15 for item in async_mod13q1_ndvi_many: pprint(item)
16 # asynchronous version of execute_all method
17 async_mod13q1_ndvi_all = provider.async_execute_all(
18     product= Products.MOD13Q1,
19     latitude= 44.04780019,
20     longitude=10.35481917,
21     band= Bands.MOD13Q1.NDVI,
22     km_above_below=0,
23     km_left_right=0
24 )
25 for item in async_mod13q1_ndvi_all: pprint(item)

```

library allows the user to query the NASA service to discover all products available for a given reference date. This is done by the *available_products_by_date()* method, which returns the list of all products along with their availability. This allows users to optimize the pipeline for data download by limiting requests to available products.

In case *nasawebservice* encounters server connection errors, client errors due to incorrect syntax, or is incapable of performing the request, an error message is produced in JSON format. The error message contains the error status code, based on the list of HTTP status codes of the Internet Assigned Numbers Authority (IANA), through which the user can decide the error handling action.

Listing 5. Snippet of *nasawebservice* accessory methods.

```

1 from nasawebservice import *
2 from pprint import pprint
3 provider = NasaWebServiceAPIProvider(base_url="https://modis.ornl.gov")
4 # Get last NDVI pixel value of last MOD13Q1 product
5 last_product = provider.last_product(
6     product= Products.MOD13Q1,
7     latitude= 44.04780019,
8     longitude=10.35481917,
9     band= Bands.MOD13Q1.NDVI,
10    km_above_below=0,
11    km_left_right=0
12 )
13 pprint(last_product)
14
15 # Check if MOD13Q1 product is present in 2019-06-26
16 check = provider.is_present(
17     product= Products.MOD13Q1,
18     latitude= 44.04780019,
19     longitude=10.35481917,
20     reference_date="A2019305"
21 )
22 print(f"Check MOD13Q1 in date 2019-06-26: {check}")
23
24 # Find all available products in 2020-05-24
25 product_available = provider.available_products_by_date(
26     latitude= 44.04780019,
27     longitude=10.35481917,
28     reference_date="A2020337"
29 )
30 pprint(product_available)

```

2.3. General architecture and workflows

In this section we illustrate *nasawebservice* capabilities to implement a cloud-based solution for MODIS data ingestion. According to this workflow, a user can combine *nasawebservice* with other open-source services on local and cloud resources without the cost of buying any cloud service.

This architecture was tested to collect the NDVI (Normalized Difference Vegetation Index) band of the MOD13Q1 (MODIS/Terra Vegetation Indices 16-Day L3 Global 250 m SIN Grid) product. The MOD13Q1 are generated every 16 days at 250 m (m) spatial resolution. The MODIS NDVI vegetation index is the most used vegetation index for agricultural and ecological studies, as it is a proxy of leaf area index and

vegetation status (e.g. Bajocco et al., 2022; Pettorelli et al., 2005; Running et al., 2004; Tuck et al., 2014).

The procedure described here is reproducible as we used 3000 sites randomly selected from the center point locations of maize (*Zea mays* L.) fields in Veneto (Italy), derived from the ISTAT (Italian National Institute of Statistics) agricultural crops database of 2019 (the whole point dataset is publicly available at <https://doi.org/10.6084/m9.figshare.23662488.v1>). Using an Infrastructure as a Service (IaaS) service, a Linux Virtual Machine (VM) has been created and deployed through the Microsoft Azure portal, a web management interface for all Azure cloud-computing services. A general purpose VM DSv2, ideal for testing and development, was instantiated with an Ubuntu Server 20.04 LTS installed as operating system. The general VM settings included the allocation of

two virtual Intel Xeon CPUs (i.e., number of cores allocated to an Azure VM), 7 GB of virtual RAM (i.e., maximum amount of memory that a VM can use at any point in time) and 30 GB of virtual SSD disk cache, for a total monthly average cost of 85 euro. Notice that the final service price could be different depending, for instance, on the type of Microsoft Customer Agreement (MCA), the additional resources employed, the data residency region choice, and the virtual machine usage time. Using the Azure pricing calculator tool, a user can estimate the cloud costs for new Azure deployments, or variations of existing workloads.

Once the VM is deployed, a Python 3.9 release was installed on the machine. The Python interpreter has been responsible for parallel execution of two *nasawebservice* processes, a static process, and a periodic process. As best practice, each process is contained in a separate virtual environment, so the dependencies are isolated from the system, and reciprocally.

The static process, using the *async execute all()* method exposed by

the *NasaWebServiceAPIProvider* class, collects all historical data for a given product, band, and location. The running session of the static process could be start through a multiplexer terminal such as GNU Screen or Tmux. The periodic process makes daily requests to the MODIS/VIIRS web service in order to collect the latest MODIS data. To perform this task, first this process uses the *is_present()* method with the required date as argument. If the returned reply is positive (i.e., True), the second method *last_product()* is executed. Although the 16-day imagery are uploaded into the archive every 16 days, delays in archiving may occur. To limit this drawback, each execute session has been set as daily. The simplest solution to schedule a periodic execution in a Linux distribution is the crontab scheduler. For both applications, any encountered error during the execution is collected in a dedicated log collector registry. The simplified extract of the code used in the two applications is provided in Listing 6. The code was executed for product MOD13Q1 and NDVI band, on April 27th 2022 and for the location centered at the following coordinates: latitude 44.04780019 N, longitude 10.35481917 E.

Listing 6. Snippet of *nasawebservice* package.

```

1 from nasawebservice import *
2 # Create a MODIS data provider
3 provider = NasaWebServiceAPIProvider(base_url="https://modis.ornl.gov")
4 # STATIC PROCESS: retraive all data in the NASA's archive
5 archive = provider.async_execute_all(
6     product= Products.MOD13Q1,
7     latitude= 44.04780019,
8     longitude=10.35481917,
9     band= Bands.MOD13Q1.NDVI,
10    km_above_below=0,
11    km_left_right=0
12 )
13 for item in archive: print(item)
14 # PERIODIC PROCESS: get the last data if present
15 if provider.is_present(Products.MOD13Q1, 44.04780019, 10.35481917, "A2022081"):
16     last = provider.last_product(
17         product= Products.MOD13Q1,
18         latitude= 44.04780019,
19         longitude=10.35481917,
20         band= Bands.MOD13Q1.NDVI,
21         km_above_below=0,
22         km_left_right=0
23     )
24     print(last)

```

Each ingested data set has to be archived into a storage service, which has been implemented with MongoDB database. MongoDB is an open-source document-oriented NoSQL database that uses JSON- like documents and allow a schemeless data model. Despite the traditional relational database management systems, MongoDB appears to be better suited to managing large volumes of remote sensing data and to be accessed by agricultural models (Wang et al., 2019).

According to the single-responsibility principle (Martin et al., 2003), *nasawebservice* does not automatically treat operations as transactions and needs solution for interfacing with MongoDB such as MongoEngine, an Object-Document Mapper (ODM) written in Python for working with

MongoDB (Reniers et al., 2019). This solution was implemented inside the *DataHarvester* class. In the domain of Remote Sensing, the *DataHarvester* component is a set of synchronous or asynchronous processes which is capable to detect new satellite images from the space agency database. Once a new image is found, the component downloads the data using the internal scripts (Bellini et al., 2021). Different implementations of the *DataHarvester* class can target other databases, ensuring flexibility of use of *nasawebservice* for different user needs. For instance, a client could use different open-source NoSQL frameworks such as HBase introduced by Google Bigtable, Elasticsearch developed by Java on Apache Lucene, or eventually one of the many SQL database solutions. Whichever storage solution chosen by the user, the stored MODIS data provides support, together with other data types, for agricultural models, data analytics, dashboard, and thus for providing services for farmers and public and private decision makers. The scheme of the cloud-based solution is shown in Fig. 1.

Several factors can affect the download speed rate of *nasawebservice*, e.g., the overall network congestion, the server traffic at the ORNL DAAC MODIS web service, the size of data set being downloaded, and the features of the user's terminal device. In order to estimate the perfor-

mance of *last_product()* and *async execute_all()*, we replicated the data download on 3000 sites, setting the parameters “km_above_below” and “km_left_right” to 0 km (i.e., the single cell of the location), to 50 km (i.e., a matrix of 401 × 401 cells), and to 100 km (i.e., 801 × 801 cells). To retrieve the last product, the average download speed of our cloud solution was 1.819 (± 0.558) seconds for 0 km, 2.231 (± 0.226) seconds for 50 km, and 3.038 (± 0.340) seconds for 100 km. In term of computational time, the most onerous operation is the acquisition of all archive data for a given location. To download all 510 sets for each location (from 2000 to 02-18 to 2022-04-07), the average download speed was, in seconds, 73.894 (± 6.52) for 0 km, 152.202 (± 43.866) for 50 km, and 421.085 (± 95.796) for 100 km. The time performance was computed using the *perf_counter()* stopwatch method of the Python built-in module *time*.

2.4. Envisaged use of *nasawebservice* for research and applications in agriculture and ecology

Releasing a specialized Python package for streamlined download and pre-processing of vegetation indices and satellite imagery from the MODIS satellite offers significant benefits for various agriculture and ecological studies, and for operational near-real time monitoring purposes. Among possible realizations using *nasawebservice*, we envisage its use in Smart Farming for precision agriculture, as the package could enable farmers and agronomists to effortlessly access and process MODIS data to monitor crop health and detect anomalies (Gnilke and Sanders, 2022), and optimize irrigation (United States Geological Survey, 2023) and fertilization strategies (e.g., Matlok et al., 2021). By automating the retrieval and preprocessing steps, the package empowers users to make timely and informed decisions for improved crop management, potentially leading to higher yields and resource efficiency. In ecological studies, the package provides researchers with a powerful tool to analyze vegetation dynamics and ecosystem health. For instance, it can facilitate the calculation of vegetation indices to assess vegetation biomass, and vigor up to final yield. This enables ecologists to monitor and understand changes in biodiversity, detect habitat degradation (Eckert et al., 2015), and track the impact of climate change on ecosystems (Jabal et al., 2022). The package's efficient data retrieval and preprocessing capabilities save researchers valuable time and resources, allowing them to focus on analysis and interpretation of the ecological data. Furthermore, *nasawebservice* can be valuable for land cover and land use mapping. By simplifying the acquisition and preprocessing of satellite imagery, it aids in handling large amounts of images used for classifying and mapping different land cover types, such as forests, croplands, and urban areas (e.g., Ali et al., 2023). This information is crucial for land management, urban planning, and environmental impact assessments. Lastly, the package can benefit studies related to wildfire monitoring and management (De Angelis et al., 2012). MODIS data can be used to assess fire risk, monitor active fires, and analyze the extent of burned areas (e.g., Radočaj et al., 2022). By integrating the package into their workflow, fire management agencies and researchers can access near-real-time data, process it efficiently, and generate actionable insights for fire suppression efforts, post-fire recovery planning, and understanding fire ecology.

3. Conclusion

In this manuscript, we provided an overview of new Python package *nasawebservice*, whose main purpose is to optimize the interface with NASA web service and reduce the pre-processing steps and related time needed to download large amounts of satellite imagery for agroecological studies. The overarching rationale for proposing this new Python package is to present a set of methods and ready-to-use functions for retrieving MODIS data from the NASA MODIS/VIIRS Land Products web service model. This package could be part of an automated data ingestion pipeline, both in local and cloud environment, being able to integrate the first part of an Extract Transform and Load process (ETL) scheme. The main advantage of using this package is that the data obtained are in JSON format (Wang, 2011). In this way it is possible to avoid all those pre-processing operations that hamper the use of satellite imagery. This advantage is due to the NASA's implementation choice for the realization of its MODIS and VIIRS data web service (Tuck et al., 2014). Therefore, based on the example of the NASA MODIS/VIIRS Land Products web service model, each space agency should realize a service based on REST APIs, to integrate into the standard service of downloading raster images. This will bring undoubted benefits for the implementation of automated processing data systems.

Beyond the proposed IaaS with a VM, the *nasawebservice* could be implemented into a serverless computing solution such as Azure Function or AWS Lambda, and then a user could define the triggers or events that will execute the code. Moreover, a virtualization by Containers (e.

g., Docker or Kubernetes) could be used as an alternative to the more traditional VM. The package is being actively developed, and in the near future, we plan to further extend *nasawebservice*'s support, by adding additional methods to its set of already implemented classes. In particular, we will add data validation methods in order to carry out a data quality control before the storage operation.

The current stable source code of *nasawebservice* is available upon request to authors and a PyPI package for *nasawebservice* is also under development. For a more in-depth walkthrough for downloading and using MODIS data with *nasawebservice*, the associated docstrings by installing the package and then entering the command "*help(NasaWebServiceAPIProvider)*" into Python is available.

Funding

This research was funded by Italian Ministry of Agriculture, Agri-Digit program (DM 36503.7305.2018 of 20 December 2018).

Declaration of Competing Interest

No potential conflict of interest was reported by the author(s).

Data availability

Data are downloadable from <https://modis.gsfc.nasa.gov/data>.

References

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D.G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y., Zheng, X., 2016. TensorFlow: a system for large-scale machine learning. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp. 265–283.
- Alcantara, C., Kuemmerle, T., Prishchepov, A.V., Radeloff, V.C., 2012. Mapping abandoned agriculture with multi-temporal MODIS satellite data. *Remote Sens. Environ.* 124, 334–347.
- Ali, S., Qi, H.A., Henchiri, M., Sha, Z., Khan, F.U., Sajid, M., Zhang, J., 2023. Remote sensing strategies to monitoring land use maps with AVHRR and MODIS data over the South Asia regions. *Environ. Sci. Pollut. Res.* 30, 31741–31754.
- Amani, M., Ghorbanian, A., Ahmadi, S.A., Kakooei, M., Moghimi, A., Mirmazloumi, S.M., Moghaddam, S.H.A., Mahdavi, S., Ghahremanloo, M., Parsian, S., Wu, Q., Brisco, B., 2020. Google earth engine cloud computing platform for remote sensing big data applications: a comprehensive review. *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.* 13, 5326–5350.
- Angearu, C.V., Ontel, I., Boldeanu, G., Mihailescu, D., Nertan, A., Craciunescu, V., Catana, S., Irimescu, A., 2020. Multi-temporal analysis and trends of the drought based on MODIS data in agricultural areas, Romania. *Remote Sens.* 12 (23), 3940.
- Bacco, M., Berton, A., Ferro, E., Gennaro, C., Gotta, A., Matteoli, S., Paonessa, F., Ruggeri, M., Virone, G., Zanella, A., 2018. Smart farming: opportunities, challenges and technology enablers. In: 2018 IoT Vertical and Topical Summit on Agriculture-Tuscany (IOT Tuscany), pp. 1–6.
- Bajocco, S., Ginaldi, F., Savian, F., Morelli, D., Scaglione, M., Fanchini, D., Raparelli, E., Bregaglio, S., 2022. On the use of NDVI to estimate LAI in field crops: implementing a conversion equation library. *Remote Sens.* 14, 15. <https://doi.org/10.3390/rs14153554>.
- Bellini, P., Bologna, D., Han, Q., Nesi, P., Pantaleo, G., Paolucci, M., 2020. Data ingestion and inspection for smart city applications. In: 2020 IEEE International Conference on Smart Computing (SMARTCOMP), pp. 23–243.
- Bellini, P., Cenni, D., Mitolo, N., Nesi, P., Pantaleo, G., 2021. Exploiting satellite data in the context of Smart City applications. In: IEEE Smart World, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Internet of People and Smart City Innovation, pp. 39–406.
- Bellini, P., Cenni, D., Mitolo, N., Nesi, P., Pantaleo, G., Soderi, M., 2022. High level control of chemical plant by industry 4.0 solutions. *J. Ind. Inf. Integr.* 26, 100276.
- Bregaglio, S., Ginaldi, F., Raparelli, E., Fila, G., Bajocco, S., 2023. Improving crop yield prediction accuracy by embedding phenological heterogeneity into model parameter sets. *Agric. Syst.* 209, 103666.
- Crowley, M.A., Cardille, J.A., 2020. Remote Sensing's recent and future contributions to landscape ecology. *Curr. Landsc. Ecol. Rep.* 5, 45–57.
- De Angelis, A., Bajocco, S., Ricotta, C., 2012. Phenological variability drives the distribution of wildfires in Sardinia. *Landsc. Ecol.* 27, 1535–1545.
- Delucchi, L., 2014. pyModis: from satellite to GIS maps. In: FOSS4G 2014, September 8th–13th, 2014, Portland, Oregon, USA.
- Eckert, S., Hülsler, F., Liniger, H., Hodel, E., 2015. Trend analysis of MODIS NDVI time series for detecting land degradation and regeneration in Mongolia. *J. Arid Environ.* 113, 16–28.

- Fielding, Roy T., Richard, N.T., 2002. Principled design of the modern web architecture. *ACM Trans. Intern. Technol.* 2 (2), 115–150.
- Forget, Y., Shimoni, M., Gilbert, M., Linard, C., 2018. Complementarity between Sentinel-1 and Landsat 8 Imagery for Built-up Mapping in Sub-Saharan Africa. Preprints.
- Gnilke, A., Sanders, T.G.M., 2022. Distinguishing abrupt and gradual forest disturbances with MODIS-based phenological anomaly series. *Front. Plant Sci.* 13.
- Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., Moore, R., 2017. Google earth engine: planetary-scale geospatial analysis for everyone. *Remote Sens. Environ.* 202, 18–27.
- Hunt, J., 2019. Concurrency with AsyncIO. In: *Advanced Guide to Python 3 Programming*. Springer International Publishing, pp. 407–417.
- Jabal, Z.K., Khayyun, T.S., Alwan, I.A., 2022. Impact of climate change on crops productivity using MODIS-NDVI time series. *Civil Eng. J.* 8, 1136–1156.
- Jayaraman, P.P., Yavari, A., Georgakopoulos, D., Morshed, A., Zaslavsky, A., 2016. Internet of things platform for smart farming: experiences and lessons learnt. *Sensors* 16 (11), 1884.
- Jepsen, J.U., Hagen, S.B., Høgda, K.A., Ims, R.A., Karlsen, S.R., Tømmervik, H., Yoccoz, N.G., 2009. Monitoring the spatio-temporal dynamics of geometrid moth outbreaks in birch forest using MODIS-NDVI data. *Remote Sens. Environ.* 113 (9), 1939–1947.
- Jia, Y., Shelhamer, E., Donahue, J., Karayev, S., Long, J., Girshick, R., Guadarrama, S., Darrell, T., 2014. Caffe: convolutional architecture for fast feature embedding. In: *Proceedings of the 22nd ACM International Conference on Multimedia*, pp. 675–678.
- Justice, C.O., Vermote, E., Townshend, J.R.G., Defries, R., Roy, D.P., Hall, D.K., Salomonson, V.V., Privette, J.L., Riggs, G., Strahler, A., Lucht, W., Myneni, R.B., Knyazikhin, Y., Running, S.W., Nemani, R.R., Wan, Z., Huete, A.R., van Leeuwen, W., Wolfe, R.E., Giglio, L., Muller, J., Lewis, P., Barnsley, M.J., 1998. The Moderate resolution imaging Spectroradiometer (MODIS): land remote sensing for global change research. *IEEE Trans. Geosci. Remote Sens.* 36 (4), 1228–1249.
- Justice, C.O., Giglio, L., Korontzi, S., Owens, J., Morisette, J.T., Roy, D., Descloitres, J., Alleaume, S., Petitcolin, F., Kaufman, Y., Kaufman, Y., 2002. The MODIS fire products. *Remote Sens. Environ.* 83 (1–2), 244–262.
- Kamilaris, A., Gao, F., Prenafeta-Boldu, F.X., Ali, M.I., 2016. Agri-IoT: a semantic framework for internet of things-enabled smart farming applications. In: *2016 IEEE 3rd World Forum on Internet of Things (WF-IoT)*, pp. 442–447.
- Kawaguchi, Y., Ozawa, S., 2019. Exploring and identifying malicious sites in dark web using machine learning. In: *Gedeon, T., Wong, K., Lee, M. (Eds.), Neural Information Process. ICONIPg, Lecture Notes in Computer Science*, vol. 11955. Springer, Cham.
- Ketkar, N., Jojo, M., 2021. Introduction to pytorch. In: *Deep Learning with Python*, vol. 2021. Apress, Berkeley, CA, pp. 27–91.
- Liu, P., 2015. A survey of remote-sensing big data. *Front. Environ. Sci.* 3, 45.
- Lottes, P., Khanna, R., Pfeifer, J., Siegwart, R., Stachniss, C., 2017. UAV-based crop and weed classification for smart farming. In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3024–3031.
- Maniadaki, M., Papatheanopoulos, A., Mitrou, L., Maria, E.A., 2021. Reconciling remote sensing technologies with personal data and privacy protection in the European Union: recent developments in Greek legislation and application perspectives in environmental law. *Laws* 10 (2), 33.
- Mao, K.B., Ma, Y., Tan, X.A., Shen, X.Y., Liu, G., Li, Z.L., Chen, J.M., Xia, L., 2017. Global surface temperature change analysis based on MODIS data in recent twelve years. *Adv. Space Res.* 59 (2), 503–512.
- Martin, R.C., Newkirk, J., Koss, R.S., 2003. *Agile Software Development: Principles, Patterns, and Practices*, vol. 2. Prentice Hall, Upper Saddle River, NJ.
- Matlok, N., Basara, O., Zardzewiały, M., Gorzelany, J., Balawejder, M., 2021. Effectiveness of a complex fertilisation technology applied to Zea mays, assessed based on normalised difference vegetation index (NDVI) from Terra moderate resolution imaging spectroradiometer (MODIS). *Agriculture* 11 (8), 754. <https://doi.org/10.3390/agriculture11080754>.
- Moysiadias, V., Sarigiannidis, P., Vitsas, V., Khelifi, A., 2021. Smart farming in Europe. *Comput. Sci. Rev.* 39, 100345.
- Murphy, R.P., Ardanuy, P.E., Deluccia, F., Clement, J.E., Schueler, C., 2006. The visible infrared imaging radiometer suite. In: *Earth Science Satellite Remote Sensing*, vol. 1. Springer-Verlag, New York, USA, pp. 199–223.
- Ned, F., Borenstein, N., 1996. Multipurpose internet mail extensions (MIME) part one: Format of internet message bodies. No. rfc2045.
- Nugroho, L.E., Pratama, A.G.H., Mustika, I.W., Ferdiana, R., 2017. Development of monitoring system for smart farming using progressive web app. In: *2017 9th International Conference on Information Technology and Electrical Engineering (ICITEE)*, pp. 1–5.
- ORNL DAAC, 2018. MODIS and VIIRS Land Products Global Subsetting and Visualization Tool. ORNL DAAC, Oak Ridge, Tennessee, USA. <https://doi.org/10.3334/ORNLDAAC/1379>.
- Padalalu, P., Mahajan, S., Dabir, K., Mitkar, S., Javale, D., 2017. Smart water dripping system for agriculture/farming. In: *2017 2nd International Conference for Convergence in Technology (I2CT)*. IEEE, pp. 659–662.
- Pettorelli, N., Vik, J.O., Mysterud, A., Gaillard, J.M., Tucker, C.J., Stenseth, N.C., 2005. Using the satellite-derived NDVI to assess ecological responses to environmental change. *Trends Ecol. Evol.* 20, 503–510.
- Radočaj, D., Jurišić, M., Gašparović, M., 2022. A wildfire growth prediction and evaluation approach using Landsat and MODIS data. *J. Environ. Manag.* 304, 114351.
- Raschka, S., Patterson, J., Nolet, C., 2020. Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *Information* 11 (4), 193.
- Reniers, V., Van Landuyt, D., Rafique, A., Joosen, W.E., 2019. Object to NoSQL database mappers (ONDM): a systematic survey and comparison of frameworks. *Inf. Syst.* 85, 1–20. <https://doi.org/10.1016/j.is.2019.05.001>.
- Rocchini, D., Neteler, M., 2012. Let the four freedoms paradigm apply to ecology. *Trends Ecol. Evol.* 27, 310–311.
- Rodríguez-Mazahua, L., Rodríguez-Enriquez, C.A., Sánchez-Cervantes, J.L., Cervantes, J., García-Alcaraz, J.L., Alor-Hernández, G., 2016. A general perspective of big data: applications, tools, challenges and trends. *J. Supercomput.* 72 (8), 3073–3113.
- Running, S.W., Ramakrishna, R.N., Faith, A.H., Maosheng, Z., Matt, R., Hirofumi, H., 2004. A continuous satellite-derived measure of global terrestrial primary production. *Bioscience* 54 (6), 547–560.
- Schwager, P., Berg, C., 2021. Remote sensing variables improve species distribution models for alpine plant species. *Basic Appl. Ecol.* 54, 1–13.
- Seide, F., Amit, A., 2016. CNTK: Microsoft's open-source deep-learning toolkit. In: *2016 Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, p. 2135.
- Tamiminia, H., Salehi, B., Mahdianpari, M., Quackenbush, L., Adeli, S., Brisco, B., 2020. Google earth engine for geo-big data applications: a meta-analysis and systematic review. *ISPRS J. Photogramm. Remote Sens.* 164, 152–170.
- Tenzin, S., Siyang, S., Pobkrut, T., Kercharoen, T., 2017. Low-cost weather station for climate-smart agriculture. In: *2017 9th International Conference on Knowledge and Smart Technology (KST)*. IEEE, pp. 172–177.
- Tona, C., Bua, R., 2018. Open-source data hub system: free and open framework to enable cooperation to disseminate earth observation data and geo-spatial information. *Geophys. Res. Abstr.* 20, 3808.
- Tsalgaidou, A., Pilioura, T., 2002. An overview of standards and related technology in web services. *Distrib. Parall. Databases* 12 (2), 135–162.
- Tuck, S.L., Phillips, H.R., Hintzen, R.E., Scharlemann, J.P., Purvis, A., Hudson, L.N., 2014. MODISTools - downloading and processing MODIS remotely sensed data in R. *Ecol. Evol.* 4 (24), 4658–4668.
- United States Geological Survey, 2023. <https://www.usgs.gov/special-topics/monitoring-vegetation-drought-stress/science/modis-irrigated-agriculture> [last accessed 10/07/2023].
- Verdouw, C., Tekinerdogan, B., Beulens, A., Wolfert, S., 2021. Digital twins in smart farming. *Agric. Syst.* 189, 103046.
- Virk, A.L., Noor, M.A., Fiaz, S., Hussain, S., Hussain, H.A., Rehman, M., Ahsan, M., Ma, W., 2020. Smart farming: an overview. *Smart Village Technol.* 191–201.
- Vu, Q., Raskovic, M., Delic, V., Ronzhin, A., 2018. Trends in development of UAV-UGV cooperation approaches in precision agriculture. In: *International Conference on Interactive Collaborative Robotics*. Springer, Cham, pp. 213–221.
- Wang, G., 2011. Improving data transmission in web applications via the translation between XML and JSON. In: *2011 Third International Conference on Communications and Mobile Computing*. IEEE, pp. 182–185.
- Wang, S., Li, G., Yao, X., Zeng, Y., Pang, L., Zhang, L., 2019. A distributed storage and access approach for massive remote sensing data in MongoDB. *ISPRS Int. J. Geo Inf.* 8 (12), 533.
- Wille, M., Clauss, K., Valgur, M., Sølvesten, J., Delucchi, L., 2017. SentinelSat (v0.12): Utility to Search and Download Copernicus Sentinel Satellite Images.
- Wolfert, S., Ge, L., Verdouw, C., Bogaardt, M.J., 2017. Big data in smart farming—a review. *Agric. Syst.* 153, 6–80.
- Wulder, M.A., Loveland, T.R., Roy, D.P., Crawford, C.J., Masek, J.G., Woodcock, C.E., Allen, R.G., Anderson, M.C., Belward, A.S., Cohen, W.B., Dwyer, J., Erb, A., Gao, F., Griffiths, P., Helder, D., Hermosilla, T., Hipple, J.D., Hostert, P., Hughes, M.J., Huntington, J., Johnson, D.M., Kennedy, R., Kilic, A., Li, Z., Lymburner, L., McCorkel, J., Pahlevan, N., Scambos, T.A., Schaaf, C., Schott, J.R., Sheng, Y., Storey, J., Vermote, E., Vogelmann, J., White, J.C., Wynne, R.H., Zhu, Z., 2019. Current status of Landsat program, science, and applications. *Remote Sens. Environ.* 225, 127–147.
- Xiao, X., Boles, S., Frolking, S., Li, C., Babu, J.Y., Salas, W., Moore III, B., 2006. Mapping Paddy Rice Agriculture in South and Southeast Asia Using Multi-Temporal MODIS Images.
- Xue-Fen, W., Yi, Y., Tao, Z., Jing-Wen, Z., Sardar, M.S., 2018. Design of distributed agricultural service node with smartphone in-field access supporting for smart farming in Beijing–Tianjin–Hebei region. *Sens. Mater.* 30 (10), 2281–2293.
- Zhai, Z., Martínez, J.F., Beltran, V., Martínez, N.L., 2020. Decision support systems for agriculture 4.0: survey and challenges. *Comput. Electron. Agric.* 170, 105256.
- Zhang, X., Friedl, M.A., Schaaf, C.B., Strahler, A.H., 2004. Climate controls on vegetation phenological patterns in northern mid-and-high latitudes inferred from MODIS data. *Glob. Chang. Biol.* 10 (7), 1133–1145.
- Zhu, X.X., Tuia, D., Mou, L., Xia, G.S., Zhang, L., Xu, F., Fraundorfer, F., 2017. Deep learning in remote sensing: a comprehensive review and list of resources. *IEEE Geosci. Remote Sens. Mag.* 5 (4), 8–36.